

PROPUESTA DE PROYECTO FINAL: ALGORITMOS Y ESTRUCTURAS DE DATOS

Título del Proyecto: Implementación de un Sistema Dinámico de Evasión, Búsqueda de Rutas y Gestión de Ranking Competitivo en Entornos Matriciales (Caso de Estudio: *PUMAN - Edición Universidad Monteávila*)

Lenguaje de Programación: C Puro (Estándar C11)

1. DESCRIPCIÓN DEL PROBLEMA Y ENFOQUE TEMÁTICO

El proyecto consiste en el desarrollo de un videojuego interactivo en consola basado en la mecánica de *Pacman*, ambientado temáticamente en el campus de la Universidad Monteávila. Al iniciar el programa, el usuario debe registrar su Nombre de Jugador para dar inicio a la sesión. El jugador controlará al personaje institucional (el Pingüino de la UMA), cuyo objetivo es moverse dentro de un mapa matricial compacto para recolectar ítems académicos (créditos universitarios) y evadir a los agentes adversarios (Osos Polares).

El sistema incorpora un modelo competitivo basado en una Métrica de Puntuación y Tiempo de Juego. Cada partida —ya sea que termine en victoria al recolectar todo, o en derrota al ser atrapado— calculará un puntaje final combinando los ítems obtenidos y una bonificación o penalización según el tiempo transcurrido en segundos. Al finalizar la sesión, el programa desplegará automáticamente un Tablero de Clasificación (Ranking) que mostrará la lista histórica de todos los jugadores que han participado, ordenada de mayor a menor puntuación.

2. VINCULACIÓN Y SUSTENTACIÓN EN LOS LABORATORIOS DEL CURSO

Para asegurar la viabilidad técnica y el cumplimiento de los objetivos académicos en el lapso establecido, la arquitectura del software integrará las soluciones implementadas en las prácticas de laboratorio:

- **Modelado del Campus (Matrices):** El entorno de juego se representará mediante una matriz bidimensional dinámica de caracteres (`char mapa`). El mapa tendrá dimensiones controladas de 5x5 o 6x6 celdas. Este tamaño compacto garantiza que el procesamiento de colisiones y el refresco de la pantalla en consola se ejecuten de manera instantánea, optimizando el rendimiento global del sistema.
- **Gestión de Enemigos (Estructuras Dinámicas):** Los osos polares se administrarán a través de una Lista Enlazada Dinámica. Cada nodo guardará la posición actual (`x`, `y`) de cada enemigo, permitiendo actualizar sus movimientos en el mapa en cada ciclo del bucle principal sin desperdiciar memoria estática.
- **Persistencia de Datos y Ranking (Archivos):** Los registros de los jugadores se almacenarán en un archivo externo de texto plano (`ranking.txt`) con formato delimitado. Al terminar cada partida, el sistema cargará el historial en memoria, agregará el nuevo registro y actualizará el archivo.

3. TIPOS ABSTRACTOS DE DATOS (TAD) Y ALGORITMOS CENTRALES

A. Estructura Matricial Directa y Control de Tiempo

Al utilizar una matriz de 5x5 o 6x6, el acceso a las casillas para validar si el personaje puede avanzar o si ha recolectado un ítem se realiza en tiempo constante $O(1)$ mediante indexación directa `mapa[fila][columna]`. Paralelamente, se utilizará la librería estándar `<time.h>` para cronometrar la duración exacta de la partida y computar la métrica final de la siguiente forma:

$$\text{Puntaje Final} = (\text{Ítems Recolectados} \times 100) - (\text{Tiempo en Segundos} \times 2)$$

B. Algoritmo de Persecución: Búsqueda en Anchura (BFS) sobre Matriz

Para la inteligencia artificial de los osos polares, se implementará el algoritmo de Búsqueda en Anchura (BFS) operando directamente sobre los índices vecinos de la matriz de 5x5 o 6x6.

- **Funcionamiento:** En cada turno, el enemigo evalúa sus casillas adyacentes válidas y utiliza un TAD Cola (Queue) auxiliar para encontrar el camino más corto hacia las coordenadas del pingüino.
- **Complejidad:** Al estar el mapa acotado a un máximo de 36 celdas (6×6), la complejidad algorítmica $O(F \times C)$ (Filas por Columnas) se ejecuta en microsegundos, asegurando una fluidez perfecta en la simulación.

C. Algoritmo de Ordenamiento para el Ranking: QuickSort / MergeSort

Para generar el tablero de clasificación al final de la partida, el programa cargará el archivo `ranking.txt` en un arreglo dinámico en memoria y aplicará un algoritmo de ordenamiento eficiente.

- **Funcionamiento:** El algoritmo evaluará el campo de puntuación de cada estructura de jugador para ordenar la lista en forma descendente. En caso de empate en puntos, utilizará el tiempo de juego como criterio de desempate (menor tiempo obtiene prioridad).
- **Complejidad:** Se justifica teóricamente la eficiencia del ordenamiento bajo la notación $O(n \log n)$, cumpliendo con los estándares de optimización de la asignatura.

4. GESTIÓN DE MEMORIA DINÁMICA Y MODULARIDAD

El software mantendrá un control estricto de recursos, garantizando la liberación de la memoria reservada para el mapa, la lista enlazada de enemigos, la cola del BFS y el arreglo del ranking mediante la función `free()`.

El código se organizará en los siguientes módulos para asegurar un diseño limpio y mantenible:

mapa.h / mapa.c: Asignación de la matriz 5x5 / 6x6, renderizado en consola y control de colisiones.

agentes.h / agentes.c: TAD Lista Enlazada para los osos polares y control del movimiento del pingüino.

bfs.h / bfs.c: Implementación de la cola dinámica y cálculo de la ruta corta de persecución.

ranking.h / ranking.c: Captura de credenciales, cronómetro, lectura/escritura del archivo de texto y algoritmo de ordenamiento.

main.c: Menú principal, ciclo de juego (game loop) y despliegue del tablero de posiciones.